

# Regra de Documentação Técnica — Gravity

## Princípio fundamental

Toda feature do Gravity tem **dois públicos que precisam de documentação**: o desenvolvedor (como funciona por dentro) e o usuário/negócio (como funciona por fora). Os documentos técnicos vivem na mesma estrutura de diretórios das skills e devem estar **sincronizados** com elas e com o código. Se a skill mudou, o documento precisa mudar. Se o código mudou, o documento precisa refletir. A fonte da verdade é: **código > skill > documento**. O documento nunca contradiz os dois anteriores.

---

## Estrutura de diretórios

Os documentos técnicos espelham a estrutura das skills, adicionando uma pasta `docs/` dentro de cada domínio:

```
skills/  
  governanca/  
    lei/  
      ddd-nomenclatura/  
        SKILL.md          ← regras para o agente  
        docs/  
          guia-nomenclatura.md ← doc técnico para dev  
          glossario-campos.md  ← doc de referência para dev + negócio  
      api-design/  
        SKILL.md  
        docs/  
          guia-api-rest.md     ← doc técnico para dev  
          catalogo-endpoints.md ← doc de referência para dev + negócio  
      seguranca/  
        autenticacao/  
          SKILL.md  
          docs/  
            fluxo-autenticacao.md ← doc técnico para dev  
            permissoes-e-acessos.md ← doc de negócio para usuário  
      [produto]/  
        [feature]/  
          SKILL.md  
          docs/  
            como-funciona-[feature].md ← doc técnico para dev  
            regras-de-negocio-[feature].md ← doc de negócio para usuário
```

**Regra:** a pasta `docs/` é irmã do `SKILL.md`, nunca filha. A skill é para o agente, o doc é para humanos.

---

## Tipos de documento

### Tipo 1 — Doc Técnico (público: desenvolvedor)

Explica **como o sistema funciona por dentro**. Quem lê: dev que vai dar manutenção, dev novo fazendo onboarding, agente que precisa de contexto.

#### Estrutura obrigatória:

```
markdown
```

```
# [Nome da Feature/Módulo]
```

```
## Visão geral
```

```
[1 parágrafo: o que é, para que serve, onde mora no monorepo]
```

```
## Arquitetura
```

```
[Diagrama ou descrição das camadas envolvidas: Frontend → API → Service → Prisma]
```

```
[Listar os arquivos-chave com caminho completo no monorepo]
```

```
## Fluxo principal
```

```
[Passo a passo numerado do caminho feliz, com arquivo responsável em cada passo]
```

```
## Entidades e relações
```

```
[Tabela: Entidade | Tabela PostgreSQL (DDD) | Model Prisma (DDD) | Relação]
```

```
## Regras de negócio implementadas no código
```

```
[Lista numerada: cada regra com WHERE está implementada (arquivo + função)]
```

```
## Endpoints relacionados
```

```
[Tabela: Método | Rota DDD | Explicação | Middleware | Zod Schema]
```

```
## Dependências
```

```
[Serviços externos, filas, crons, outros módulos que este consome ou que consomem es
```

```
## Referências
```

```
[Links para: skill correspondente, aba da planilha DDD, outros docs relacionados]
```

### Tipo 2 — Doc de Negócio (público: usuário/stakeholder/PM)

Explica **como a feature funciona do ponto de vista do usuário**. Quem lê: PM, suporte, cliente, QA, stakeholder. Zero código, zero jargão técnico.

#### Estrutura obrigatória:

```
markdown
```

## # [Nome da Feature – visão de negócio]

### ## O que é

[1-2 parágrafos em linguagem simples: o que faz, para quem, qual problema resolve]

### ## Como funciona

[Passo a passo do fluxo do USUÁRIO, não do sistema. Com screenshots se possível]

### ## Regras de negócio

[Lista numerada das regras que o usuário precisa saber:]

[- Quem pode fazer o quê (permissões em linguagem humana)]

[- Limites (máximo de X, prazo de Y)]

[- Comportamentos automáticos (quando Z acontece, o sistema faz W)]

[- Exceções e casos especiais]

### ## Campos e seus significados

[Tabela: Campo (nome na tela) | O que significa | Obrigatório? | Valores possíveis]

### ## Status e ciclo de vida

[Se a entidade tem estados: diagrama ou tabela mostrando os estados e transições per

### ## Perguntas frequentes

[3-5 perguntas que suporte/PM costuma ter]

### ## Referências

[Links para: doc técnico correspondente, outros docs de negócio relacionados]

## Tipo 3 — Doc de Referência (público: ambos)

Tabelas de consulta rápida. Quem lê: dev para copiar um nome, PM para conferir um valor.

**Exemplos:** glossário de campos, catálogo de endpoints, tabela de enums, mapa de permissões.

**Estrutura:** livre, mas sempre com tabela e busca fácil. Header + tabela. Sem prosa longa.

## Regras

1. **Todo SKILL.md tem pelo menos 1 doc técnico associado.** Se a skill existe, o doc técnico deve existir na pasta `docs/` irmã. Se não existe, é uma lacuna a ser preenchida.
2. **Linguagem separada por público.** Doc técnico pode ter código, caminhos de arquivo, nomes de tabela DDD. Doc de negócio nunca tem código — traduzir tudo para linguagem humana.
  - Doc técnico: "O campo `status_pedido` aceita os valores `DRAFT`, `SUBMITTED`, `APPROVED`"

- Doc de negócio: "O pedido passa por três fases: Rascunho → Enviado → Aprovado"
3. **Nomes DDD sempre.** Todo nome de campo, tabela, enum, rota citado no doc técnico deve usar o nome DDD (conforme planilha). Nunca o nome antigo. Se o doc cita `createdAt` ao invés de `data_criacao_pedido`, está desatualizado.
  4. **Arquivo-chave sempre com caminho completo.** Nunca cite "o service de pedido" — cite `servicos-global/tenant/processos-core/server/src/services/pedido.service.ts`. O dev precisa abrir o arquivo, não adivinhar.
  5. **Regras de negócio rastreáveis.** Toda regra de negócio no doc deve ter correspondência: no doc técnico, com o arquivo onde está implementada; no doc de negócio, com a regra técnica que a suporta. Se uma regra existe no doc de negócio mas não no código, é especulação — marcar como `⚠ Não implementada`.
  6. **Endpoints sempre no formato DDD.** Tabelas de endpoints usam a rota DDD (aba `2. ddd_api`), não a rota atual. Se a rota ainda não foi migrada, mostrar ambas com indicação: `atual: /api/v1/agenda/ → DDD: /api/v1/agendas`
  7. **Diagramas em Mermaid.** Fluxos, arquiteturas e ciclos de vida usam blocos Mermaid renderizáveis no markdown. Não usar imagens estáticas para diagramas de fluxo — eles ficam desatualizados.
  8. **Sem doc órfão.** Todo documento deve ser referenciado por pelo menos um outro doc ou skill. Se ninguém aponta para ele, ninguém vai encontrá-lo.

---

## Naming dos arquivos

```
docs/  
  guia-[assunto].md           ← doc técnico (como fazer / como funciona)  
  regras-de-negocio-[feature].md ← doc de negócio (regras para usuário)  
  glossario-[dominio].md      ← referência (tabela de consulta)  
  catalogo-[tipo].md          ← referência (lista completa)  
  fluxo-[processo].md         ← doc técnico (passo a passo com diagrama)  
  permissoes-[modulo].md      ← doc de negócio (quem pode o quê)
```

Sempre kebab-case, sempre em PT-BR, sempre descritivo.

---

## Conversão: Skill existente → Documentos

| Passo | Ação                                | Resultado                      |
|-------|-------------------------------------|--------------------------------|
| 1     | Ler o <code>SKILL.md</code> inteiro | Entender o domínio e as regras |

---

| Passo | Ação                                                | Resultado                                |
|-------|-----------------------------------------------------|------------------------------------------|
| 2     | Ler o código dos arquivos citados na skill          | Entender a implementação real            |
| 3     | Comparar skill com código                           | Identificar divergências                 |
| 4     | Criar docs/ como irmã do SKILL.md                   | Estrutura de diretório                   |
| 5     | Escrever doc técnico usando dados da skill + código | guia-[assunto].md ou fluxo-[processo].md |
| 6     | Extraír regras de negócio do código + skill         | regras-de-negocio-[feature].md           |
| 7     | Cruzar endpoints do doc com aba 2. ddd_api          | Garantir rotas DDD                       |
| 8     | Cruzar nomes do doc com aba 1. ddd_campos           | Garantir naming DDD                      |

Exemplo completo

Skill: skills/seguranca/autenticacao/SKILL.md

skills/seguranca/autenticacao/  
SKILL.md ← regras para o agente  
docs/  
fluxo-autenticacao.md ← doc técnico  
permissoes-e-acessos.md ← doc de negócio

fluxo-autenticacao.md (trecho):

markdown

## # Fluxo de Autenticação

### ## Visão geral

O Gravity usa Clerk como provedor de identidade. Toda requisição autenticada carrega um JWT que é validado no middleware antes de chegar ao controller.

### ## Arquitetura

Frontend (Zustand) → API Client (Axios + interceptor Clerk) → Route → clerk-auth.ts (valida JWT) → Controller → Service → Prisma

### ## Fluxo principal

1. **Frontend** – `servicos-global/store/useMe.ts` dispara GET /api/v1/me ao montar
2. **API Client** – `servicos-global/api/configurador-client.ts` injeta Bearer token
3. **Route** – `configurador/server/src/routes/me.routes.ts` recebe e encaminha
4. **Middleware** – `configurador/server/src/middlewares/clerk-auth.ts` valida JWT,
5. **Controller** – `configurador/server/src/controllers/me.controller.ts` orquestra
6. **Service** – `configurador/server/src/services/user.service.ts` faz prisma.user.
7. **Response** – JSON com id\_usuario, id\_organizacao, nome\_usuario, permissoes

### ## Endpoints relacionados

| Método | Rota DDD   | Explicação                                                | Middleware |
|--------|------------|-----------------------------------------------------------|------------|
| GET    | /api/v1/me | Retorna dados do usuário autenticado com org e permissões | aut        |

permissoes-e-acessos.md (trecho):

markdown

## # Permissões e Acessos

### ## O que é

O Gravity controla o que cada pessoa pode ver e fazer dentro da plataforma. Cada usuário tem um nível de acesso (patente) que define quais telas e ações estão disponíveis para ele.

### ## Níveis de acesso

| Nível       | Quem é              | O que pode fazer                                    |
|-------------|---------------------|-----------------------------------------------------|
| ---         | ---                 | ---                                                 |
| Super Admin | Equipe Gravity      | Tudo – acessa todas as organizações                 |
| Admin       | Dono da organização | Gerencia usuários, workspaces, configurações        |
| Master      | Gerente/líder       | Cria e edita pedidos, processos, relatórios         |
| Standard    | Operador            | Visualiza e executa tarefas do dia a dia            |
| Fornecedor  | Parceiro externo    | Acessa apenas portais e cotações direcionadas a ele |

### ## Regras de negócio

1. Um usuário só pode acessar dados da organização à qual pertence
2. Um Admin pode convidar novos usuários e definir seus níveis
3. Um Standard não pode deletar pedidos – apenas Admin e Master
4. Fornecedores só veem cotações onde foram convidados como participantes

## Checklist de validação

Antes de considerar um documento pronto:

- ☐ Está na pasta `docs/` irmã do `SKILL.md` correspondente?
- ☐ O tipo está correto? (técnico para dev, negócio para usuário, referência para ambos)
- ☐ Doc técnico: todos os caminhos de arquivo estão completos?
- ☐ Doc técnico: todos os nomes usam DDD (cruzar com planilha)?
- ☐ Doc técnico: endpoints usam rota DDD (cruzar com aba `2. ddd_api`)?
- ☐ Doc de negócio: zero código, zero jargão técnico?
- ☐ Doc de negócio: regras de negócio são as mesmas implementadas no código?
- ☐ Diagramas de fluxo estão em Mermaid (não imagem estática)?
- ☐ O documento referencia a skill e/ou outros docs (não é órfão)?
- ☐ As regras de negócio do doc batem com o código? (se não, marcar ⚠ Não implementada)